

Agent 接口契约与编排骨架（赛道2）

本文档是给队友对接用的基准：定义编排 Agent 与各子 Agent 之间的输入/输出字段、共享状态结构，以及 Dify Workflow 里骨架节点该怎么连。具体每个 Agent 内部 prompt/技能实现不在本文档范围内（由负责该 Agent 的队友决定），本文档只锁**接口形状**。

参考架构见 [agent-architecture.md](#)。

一、共享状态 Schema

编排 Agent 维护，所有子 Agent 只读或按需回写其中一部分字段，不各自私有一份。

```
{
  "user_id": "string",
  "persona": {
    "scenario": "business_trip | vacation | family | solo_adventure",
    "pace": "packed | balanced | relaxed",
    "taste": ["string"],
    "budget_tier": "low | mid | high",
    "vector": [0.0]
  },
  "trip_state": {
    "trip_id": "string",
    "locations": [
      {
        "name": "string",
        "lat": 0.0,
        "lng": 0.0,
        "time_window": "string",
        "confirmed": false
      }
    ],
    "bookings": [
      {
        "type": "hotel | ticket | transport",
        "provider": "string",
        "status": "pending | confirmed | cancelled",
```

```
        "price": 0.0
      }
    ],
    "last_updated": "ISO8601"
  }
}
```

字段说明与谁读写：

字段	谁写	谁读	备注
persona.scenario	编排 Agent（对话中识别 / onboarding 问卷）	达人、行程、OTA	场景化，不同 trip 可能不同，见 4.1
persona.pace	同上	达人、行程	决定路线密度/推荐风格
persona.budget_tier	同上	OTA/酒店、达人	硬过滤，不是加权项
persona.vector	反馈闭环校准（见 agent-architecture.md 4.4）	达人（两阶段检索第一阶段）	embedding，具体模型由队友定
trip_state.*	编排 Agent 汇总子 Agent 结果后写入	行程、OTA、异常应变	单一事实来源，避免子 Agent 各自维护副本

二、各 Agent 输入/输出契约

编排 Agent（主 Workflow，非“被调用”的子节点）

- 输入：用户消息（自然语言）+ 当前 共享状态
- 输出：更新后的 共享状态 + 面向三面板的结构化响应：

```
{
  "chat_reply": "string",
  "community_panel": [ /* 见达人 Agent 输出 */ ],
  "map_panel": { /* 见行程 Agent 输出 */ }
}
```

- 职责：意图识别 → 决定调用哪些子 Agent（可并行/可顺序，见第三节）→ 汇总结果、写回共享状态 → 生成最终回复

达人 / 内容 Agent

	字段	说明
输入	<code>persona.vector</code> , <code>persona.budget_tier</code> , <code>location_hint</code> , <code>top_k</code>	<code>location_hint</code> 是关键词或地理范围
输出	<code>recommendations: [{place, time_slot, avg_cost, rating, similarity_score, avoid_tips, verified_trip: bool, caption, images: [string]}]</code>	<code>similarity_score</code> 是两阶段检索里"和你相似的人"的相似度百分比, 用于前端"3位相似度92%的人去过"展示; <code>caption / images</code> 是帖子图文内容, 可选, 见第五节存储格式

行程 / 路线 Agent

	字段	说明
输入	<code>locations: [{name, lat, lng}]</code> , <code>persona.pace</code> , <code>time_budget</code>	
输出	<code>route: [{order, place, eta, transport_mode, duration_min}]</code>	直接可喂给地图面板画线

OTA / 酒店 Agent

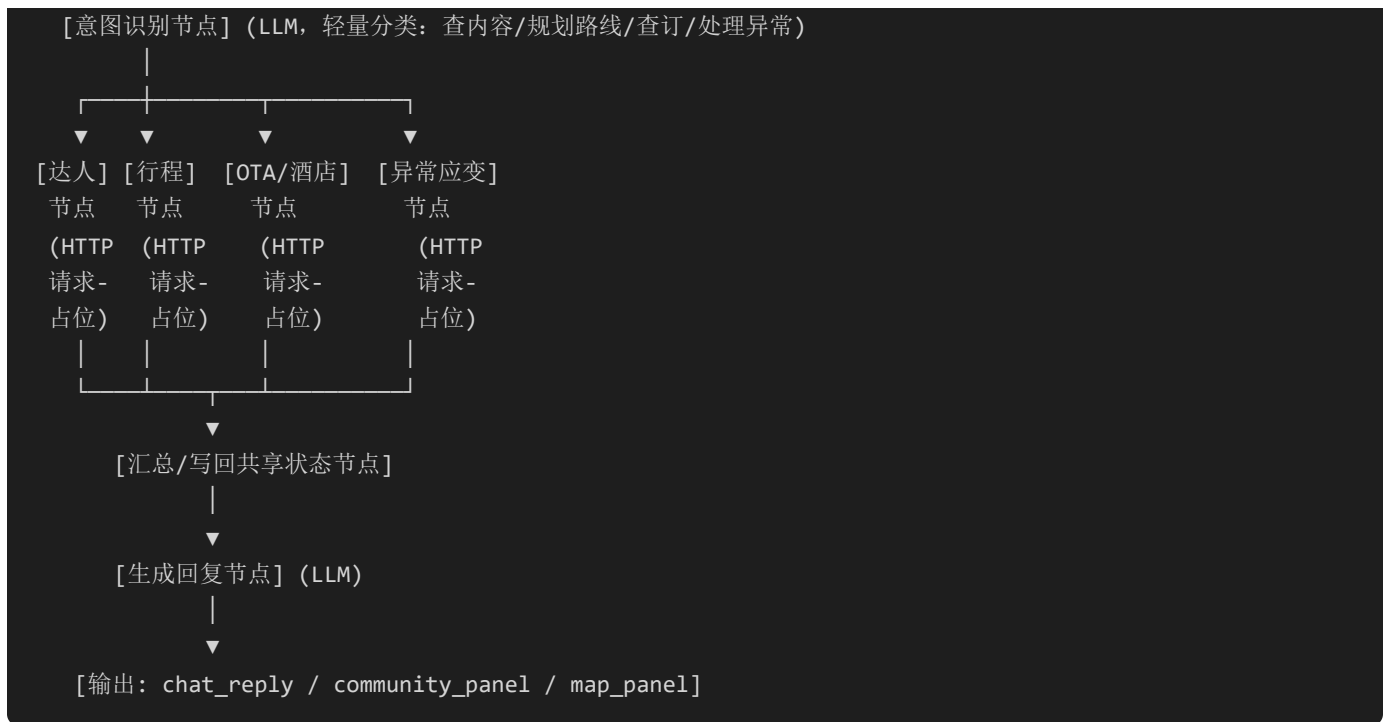
	字段	说明
输入	<code>budget_tier</code> (硬过滤) , <code>location</code> , <code>date_range</code> , <code>category</code>	酒店 Agent 与 OTA Agent 各自独立调用, 不合并 (利益不一致)
输出	<code>candidates: [{name, price, inventory, cancel_policy, rating, provider_type: "hotel" "ota"}]</code>	

异常应变 Agent

	字段	说明
输入	<code>trip_state</code> (当前行程) , <code>event_type: weather flight_delay ticket_soldout</code> , <code>event_detail</code>	由外部事件触发 (webhook/定时轮询, 可用 n8n) , 不是用户对话直接触发
输出	<code>needs_replan: bool</code> , <code>suggested_adjustment: string</code> , <code>affected_locations: [string]</code>	触发后交回编排 Agent 决定是否真的重新规划

三、编排骨架：Dify Workflow 节点拓扑





- 意图识别节点决定实际调用哪几个分支（不是每次都全调，比如纯聊天问路线不需要调 OTA）。
- 达人/行程/OTA 三个节点之间**没有依赖关系时可以并行调用**（Dify Workflow 支持并行分支），异常应变节点通常独立触发（外部事件驱动），不在用户对话主链路里。
- 第1步阶段，四个子节点先用 **HTTP Request 节点指向一个 mock server（或 Dify 自带的"代码执行"节点直接 return 固定 JSON）**，返回符合上面契约字段的假数据 —— 目的是验证"骨架跑得通、字段传递没问题"，不用等队友把每个 Agent 实现完。

四、给你现在（骨架阶段）的具体动作

1. 在 Dify 建一个 **Chatflow / Workflow** 类型 App，命名如"编排 Agent - 主流程"。
2. 在"变量"里按第一节的 Schema 建共享状态变量（Dify 支持 Object/Array 类型变量）。
3. 加一个 LLM 节点做意图识别，用一个轻量模型（对齐你架构文档里"模型档位"的设计）。
4. 加 4 个"代码执行"节点（Python/JS，Dify 内置），每个节点内容就是 `return` 一段写死的 JSON，字段照抄本文档第二节的输出契约——这就是"占位子 Agent"。
5. 用条件分支把意图识别的结果接到对应占位节点，跑几轮测试消息，确认字段能从占位节点一路传到最终输出没有丢字段/类型错。
6. 骨架跑通后，把本文档甩给队友，让他们把占位节点换成真实的 Agent（可以是各自独立的 Dify App，编排 Agent 用 HTTP 请求节点调用），只要不改字段名和结构，你的主流程不用动。

更新：第四节的 Dify 落地方式后来放弃了（Dify 云端遇到 Cloudflare 拦截，调试成本太高），改成直接写 Python 骨架，见 `orchestrator/main.py` ——节点拓扑、字段契约本身没变，只是从 Dify 的可视化节点换成了 Python 函数，队友接手时对照的还是第二节的字段契约。

五、达人社区帖子存储格式（图文）

达人社区支持图文帖子（文字 + 多图），对应实现在 `orchestrator/store.py`。队友插入模拟数据时按这个格式来。

存储方式

- **结构化字段 + 文字正文**：存进 Chroma 向量库的 metadata（`add_post()` 的 `content` 参数）。
- **图片**：不进向量库（向量库不适合存二进制大文件），本地落盘在 `orchestrator/data/images/<post_id>/` 下；`content["images"]` 里只存相对路径字符串列表，不存图片本身。用 `save_image(post_id, filename, image_bytes)` 写图，会自动落盘并返回该相对路径，直接塞进 `images` 列表即可。

content 字段完整列表

字段	类型	说明
place	string	地点
time_slot	string	时段
avg_cost	number	人均消费
rating	number	评分
avoid_tips	string	避坑提示，可选
verified_trip	bool	是否为系统留存的已验证行程（见 agent-architecture.md 4.3）
caption	string	帖子正文文字，可选
images	list[string]	图片相对路径列表，可选；由 <code>save_image()</code> 生成，不要手填绝对路径

队友插入模拟数据的数据流

```
1. 调 save_image("post-001", "1.jpg", <图片二进制>)
   → 返回 "images/post-001/1.jpg", 图片被写到 orchestrator/data/images/post-001/1.jpg

2. 调 add_post(
    post_id="post-001",
    persona_vector=[...],          # 人格向量，人格系统没定之前可以先随便造几个测试向量
    content={
        "place": "西湖", "time_slot": "上午", "avg_cost": 120, "rating": 4.7,
        "caption": ".....", "images": ["images/post-001/1.jpg"],
    },
)

3. 调 query_similar_posts(query_vector, top_k=5) 验证能查出来,
```

返回结果里 `images` 字段会自动还原成路径列表。

4. 前端/展示层要读图片实际内容时，用 `resolve_image_path(images[0])` 拿本地绝对路径。

已知限制 / 以后要换的地方

- 图片存本地文件系统，只在当前开发机上可见。以后部署到 ModelScope Studio 或给评委远程访问 demo 时，需要把 `save_image()` / `resolve_image_path()` 换成对象存储（比如 OSS/CDN）的上传/取 URL 逻辑，`content["images"]` 里存的字符串到时候就是完整 URL 而不是本地相对路径——上层调用方式不用变。
- `persona_vector` 目前由调用方随意提供（人格系统还没定义），队友测试阶段可以先用随机向量占位，等人格系统定下来后统一换成真实生成的向量。